Domain-Driven Design (DDD)

Como modelar software a partir do domínio do negócio, reduzindo ambiguidades e organizando sistemas complexos.

1. O que é DDD?

Domain-Driven Design, ou DDD, é uma abordagem de desenvolvimento de software popularizada por Eric Evans. Seu ponto central é simples: quando o software resolve um problema de negócio complexo, o modelo do software deve refletir esse negócio de forma explícita.

DDD não é uma arquitetura pronta, um framework ou uma estrutura de pastas. Ele é um conjunto de princípios, padrões e práticas para aproximar especialistas do negócio e desenvolvedores, criando uma representação compartilhada do domínio.

Em vez de começar pelo banco de dados, pelas telas ou pelos endpoints, o time procura primeiro compreender conceitos, regras, processos, invariantes, eventos e limites do negócio. Depois, transforma esse conhecimento em modelos e código.

A ideia central

Um sistema de vendas, por exemplo, não é apenas um conjunto de tabelas como customers, orders e products. Existem regras: quando um pedido pode ser cancelado? O que significa um pagamento aprovado? Quando o estoque é reservado? Quem é responsável por cada decisão? DDD tenta tornar essas regras parte explícita do modelo.

DDD é especialmente útil quando o problema de negócio é complexo e muda com frequência. Para um CRUD simples, muitos dos seus padrões podem ser desnecessários.

Objetivos principais

- Entender o domínio: descobrir como o negócio realmente funciona.
- Criar uma linguagem comum: reduzir traduções e ambiguidades entre negócio e tecnologia.
- Definir limites: evitar que um único modelo tente representar toda a empresa.
- Proteger regras importantes: manter decisões de negócio próximas dos conceitos aos quais pertencem.
- Permitir evolução: mudar partes do sistema com menor impacto sobre as demais.

2. Domínio, subdomínios e linguagem ubíqua

Domínio é a área de conhecimento e atividade que o software pretende apoiar. Em um e-commerce, o domínio pode envolver vendas, pagamentos, logística, catálogo, estoque, atendimento e prevenção a fraude.

Subdomínios

Como o domínio inteiro costuma ser grande demais para ser tratado como uma única coisa, DDD propõe dividi-lo em subdomínios. Essa divisão é conceitual e orientada pelo negócio.

Tipo	Significado	Exemplo
Core Domain	Parte que gera diferenciação e merece maior investimento.	Motor de precificação exclusivo.
Supporting	Necessário ao negócio, mas não é sua principal vantagem competitiva.	Gestão de devoluções.
Generic	Problema comum, frequentemente resolvido por soluções existentes.	Autenticação ou envio de e-mail.

Linguagem Ubíqua (Ubiquitous Language)

É uma linguagem compartilhada entre pessoas técnicas e especialistas do domínio. Os mesmos termos usados nas conversas, regras e documentação devem aparecer, sempre que fizer sentido, no código.

Se o negócio chama uma operação de Reserva de Estoque, o código deveria preferir conceitos como StockReservation ou ReserveStock, em vez de nomes genéricos como ProcessData. O objetivo não é traduzir palavra por palavra, mas preservar o significado do domínio.

A linguagem ubíqua é viva: quando o entendimento do negócio melhora, a linguagem e o modelo também devem evoluir.

Como descobrir essa linguagem?

- Conversar com especialistas e observar os termos que eles usam.
- Questionar palavras ambíguas: “cliente” significa comprador, usuário, conta ou empresa?
- Modelar cenários, regras e eventos do negócio.
- Usar técnicas colaborativas como Event Storming para revelar processos e conceitos.

3. Bounded Context: o limite do modelo

Um dos conceitos mais importantes do DDD é o Bounded Context. Ele define uma fronteira dentro da qual um determinado modelo e sua linguagem possuem significado consistente.

A mesma palavra pode significar coisas diferentes em contextos diferentes. “Produto”, por exemplo, pode representar descrição, fotos e categorias no Catálogo; quantidade disponível e localização no Estoque; preço e condições comerciais em Vendas. Forçar todos esses significados em uma única entidade global tende a criar forte acoplamento.

Exemplo conceitual

CATÁLOGO	VENDAS	ESTOQUE
Produto	Item vendável	SKU
Descrição	Preço	Saldo
Categoria	Desconto	Reserva
Imagem	Condição comercial	Localização

Context Map

Depois de identificar contextos, é preciso entender como eles se relacionam. O Context Map documenta integrações, dependências e relações entre modelos. Um contexto pode publicar eventos, expor uma API ou adaptar o modelo de outro contexto.

Um princípio importante é evitar compartilhar indiscriminadamente o modelo interno. Cada contexto deve controlar seu próprio significado. Quando necessário, uma Anti-Corruption Layer (ACL) traduz o modelo externo para o modelo local.

Bounded Context não é sinônimo automático de microsserviço.

Um contexto pode ser implementado como módulo de um monólito modular, como um serviço independente ou até por mais de um componente. Primeiro define-se o limite conceitual; depois escolhe-se a topologia técnica adequada.

Sinais de que os limites estão ruins

- Mudanças simples exigem alterações coordenadas em muitos módulos.
- Uma entidade possui dezenas de campos usados por áreas diferentes.
- Times discordam constantemente sobre o significado dos mesmos termos.
- Eventos e APIs expõem detalhes internos em vez de contratos de negócio.
- Um contexto precisa conhecer regras que deveriam pertencer a outro.

4. Building Blocks: modelando dentro do contexto

Dentro de um Bounded Context, o DDD oferece padrões táticos para representar comportamento e regras. Eles não precisam ser usados todos ao mesmo tempo; devem existir quando tornam o modelo mais claro.

Entity

Objeto com identidade contínua ao longo do tempo. Ex.: Pedido #123 pode mudar de status e itens, mas continua sendo o mesmo pedido.

Value Object

Objeto definido por seus valores, geralmente imutável e sem identidade própria. Ex.: Dinheiro, Endereço, Intervalo de Data.

Aggregate

Conjunto de objetos tratado como uma unidade de consistência. Possui uma raiz, o Aggregate Root, que controla alterações relevantes.

Repository

Abstração para recuperar e persistir Aggregates sem espalhar detalhes de armazenamento pelo domínio.

Domain Service

Comportamento de domínio importante que não pertence naturalmente a uma única Entity ou Value Object.

Domain Event

Fato relevante que já aconteceu no domínio. Ex.: PedidoConfirmado, PagamentoAprovado, EstoqueReservado.

Aggregate e invariantes

Uma invariante é uma regra que deve permanecer verdadeira. Imagine um Pedido cuja regra diga: “um pedido enviado não pode receber novos itens”. O Aggregate deve proteger essa regra, evitando que qualquer parte do sistema altere seu estado de forma inválida.

Um bom modelo de domínio concentra comportamento, não apenas dados. Em vez de alterar diretamente order.status = shipped, uma operação como order.ship() pode verificar as regras antes de produzir a mudança.

DDD e arquitetura

DDD combina bem com arquiteturas como Hexagonal, Ports & Adapters, Clean Architecture, Vertical Slice e Monólito Modular. Entretanto, DDD e arquitetura são preocupações diferentes: DDD ajuda principalmente a modelar o domínio e seus limites; a arquitetura organiza dependências e execução técnica.

5. Como aplicar DDD na prática

DDD funciona melhor como processo de descoberta contínua, e não como uma grande modelagem feita uma única vez antes do desenvolvimento. O modelo deve evoluir junto com o entendimento do negócio.

Um caminho prático

- 1. Escolha um fluxo importante: por exemplo, “realizar pedido” ou “processar devolução”.
- 2. Converse com especialistas: descubra regras, exceções, decisões, eventos e termos.
- 3. Modele o processo: Event Storming pode ajudar a visualizar comandos, eventos, atores e políticas.
- 4. Identifique subdomínios e limites: procure mudanças de linguagem, responsabilidade, regras e ownership.
- 5. Defina Bounded Contexts: explicite qual modelo pertence a cada limite.
- 6. Modele o núcleo: use Entities, Value Objects, Aggregates e Domain Events onde agregarem clareza.
- 7. Defina integrações: contratos, APIs, eventos e ACLs entre contextos.
- 8. Implemente e aprenda: valide o modelo com casos reais e refatore quando o entendimento mudar.

Exemplo resumido

Em uma compra, o contexto Vendas confirma o Pedido e publica PedidoConfirmado. O contexto Pagamento interpreta esse fato segundo seu próprio modelo, processa a cobrança e publica PagamentoAprovado. O contexto Logística pode reagir ao evento e iniciar a preparação. Cada contexto possui responsabilidades claras e não precisa compartilhar uma gigantesca entidade “Pedido” com todos os detalhes da empresa.

Quando usar?

DDD tende a gerar mais valor em sistemas com regras complexas, múltiplas áreas de negócio, vocabulário especializado, longa vida útil e mudanças frequentes. Em aplicações pequenas e predominantemente CRUD, uma abordagem mais simples costuma ser suficiente.

Resumo final

DDD é, antes de tudo, uma forma de lidar com complexidade de negócio. Seus conceitos mais importantes são: compreender profundamente o domínio, desenvolver uma linguagem ubíqua, separar modelos por Bounded Contexts e representar regras importantes de maneira explícita. Os padrões táticos são ferramentas; o objetivo é construir um software cujo desenho acompanhe o significado e a evolução do negócio.

Pergunta-guia: “O código expressa como o negócio pensa e protege as regras que realmente importam?” Se a resposta for sim, você está próximo do espírito do DDD.